

An Internship Report

Of

Artificial Intelligence and Machine Learning

Project: Twitter Sentiment Analysis Using Logistic Regression

Submitted

To

School of Computer Science and Engineering

IILM University, Greater Noida, U.P.

In partial fulfilment of the requirement of degree of

B.Tech (Computer Science and Engineering)



By

Name of Student: SHUBHAM MISHRA

Session: 2025–2029

Internship Organization

CODEC TECHNOLOGIES PVT. LTD.

Designation: Artificial Intelligence Intern

17 July 2026 – 31 August 2026 | 45 Days

1. CANDIDATE'S DECLARATION

I, **SHUBHAM MISHRA**, student of Bachelor of Technology in Computer Science and Engineering at **IILM University, Greater Noida, U.P.**, Session **2025–2029**, hereby declare that this internship report titled **“Artificial Intelligence and Machine Learning with Twitter Sentiment Analysis Using Logistic Regression”** is a record of the learning, training activities, and practical project work completed during my internship at **Codec Technologies Pvt. Ltd.**

The report has been prepared to document the knowledge and practical exposure gained during the 45 Days internship conducted from **17 July 2026 to 31 August 2026**. It covers the fundamentals of Artificial Intelligence and Machine Learning studied during the internship and provides a detailed account of the Twitter Sentiment Analysis project implemented using Python, Natural Language Processing techniques, TF-IDF vectorization, and Logistic Regression.

I further declare that the project implementation described in this report was carried out as part of the internship learning and project activities. Technical references and external resources used to understand the concepts have been acknowledged in the references section.

Name: Shubham Mishra

Programme: B.Tech (Computer Science and Engineering)

Session: 2025–2029

Signature: _____

2. ACKNOWLEDGEMENT

I would like to express my sincere gratitude to **Codec Technologies Pvt. Ltd.** for providing me with the opportunity to undertake a 45 Days internship in the field of Artificial Intelligence. The internship gave me a structured environment in which I could study Artificial Intelligence and Machine Learning concepts while also applying them to a practical text classification problem.

I am thankful for the learning exposure provided throughout the internship. The training helped me understand that machine learning is not limited to selecting an algorithm. A practical AI solution requires a complete workflow involving problem definition, data collection, data inspection, preprocessing, feature engineering, model training, evaluation, and finally the use or persistence of the trained model.

I am particularly grateful for the opportunity to work with a large text dataset and implement a complete Natural Language Processing pipeline. The Twitter Sentiment Analysis project allowed me to apply concepts such as stop-word removal, stemming, TF-IDF feature extraction, train-test splitting, Logistic Regression, accuracy evaluation, and model saving.

I would also like to acknowledge the support of **IILM University, Greater Noida**, and the School of Computer Science and Engineering for providing the academic environment in which the internship learning could be connected with my degree studies in Computer Science and Engineering.

Finally, I am thankful to everyone who contributed directly or indirectly to my learning and helped me complete the internship activities and project documentation.

Shubham Mishra

B.Tech Computer Science and Engineering | Session 2025–2029

3. TABLE OF CONTENTS

Chapter	Title	Page
1	Candidate's Declaration	i
2	Acknowledgement	ii
3	Table of Contents	iii
4	Internship Completion Certificate and Offer Letter	1
5	Introduction	3
6	Organization Profile	4
7	Internship Overview and Objectives	5
8	Training and Learning During the Internship	7
9	Technologies and Tools Used	11
10	Project Description: Twitter Sentiment Analysis	12
11	Dataset and Data Understanding	14
12	Data Preprocessing and NLP Pipeline	16
13	Feature Extraction with TF-IDF	19
14	Model Development using Logistic Regression	21
15	Model Evaluation and Results	23
16	Prediction and Model Persistence	25
17	Applications, Limitations and Future Scope	27
18	Learning Outcomes	29
19	Conclusion	31
20	References	32

4. INTERNSHIP COMPLETION CERTIFICATE AND OFFER LETTER

4.1 Internship Offer Details

The internship offer letter issued by Codec Technologies confirms the offer of a 45 Days internship and identifies the internship designation as **Artificial Intelligence Intern**. It states that the internship was conducted in a Pan India, Hybrid mode from **17/07/2026 to 31/08/2026** and that interns reported to assigned project head(s). The offer letter also describes the program as an immersive experience intended to provide industry-level knowledge and valuable skills.

Field	Details
Student	Shubham Mishra
Organization	Codec Technologies
Designation	Artificial Intelligence Intern
Internship Type	45 Days Internship
Mode	Pan India, Hybrid
Start Date	17/07/2026
End Date	31/08/2026
Reporting	Assigned Project Head(s)
NCS ID	E19E86-0116588288923
AICTE ID	CORPORATE6759d549ce59e1733940553
Date of Issue	17/07/2026

The offer letter further states that assigned projects and training tasks were to be completed through the organization's platform and email. It also mentions that exceptional interns could qualify for a scholarship award based on weekly performance reviews conducted by the project supervisor.

5. INTRODUCTION

Artificial Intelligence (AI) has become an important area of Computer Science because of its ability to automate tasks that traditionally require human reasoning, pattern recognition, language understanding, or decision-making. Modern AI systems are increasingly built around data, algorithms, and computational models that learn useful patterns from examples.

Machine Learning (ML) is one of the most widely used approaches within Artificial Intelligence. Instead of explicitly programming every possible rule, a machine learning system is trained using data so that it can identify relationships between input features and desired outputs. This approach is particularly useful when the rules of a problem are complex or difficult to write manually.

Natural Language Processing (NLP) is a branch of AI concerned with human language. Social media platforms, online reviews, support systems, news websites, and messaging platforms produce enormous quantities of textual information. Automatically understanding this information creates opportunities for sentiment analysis, spam detection, topic classification, search, recommendation, and customer feedback analysis.

The internship at Codec Technologies Pvt. Ltd. provided a broad introduction to AI and ML and allowed these concepts to be applied in a practical project. The selected project, Twitter Sentiment Analysis, focuses on automatically identifying whether a tweet expresses a positive or negative sentiment.

The project uses the Sentiment140 dataset, which contains approximately 1.6 million tweets. The raw textual information is preprocessed using regular expressions, lowercase conversion, stop-word removal, and Porter stemming. The resulting text is converted into numerical features using TF-IDF, after which a Logistic Regression classifier is trained and evaluated.

The overall project therefore represents a complete introductory machine learning pipeline: data acquisition, data inspection, preprocessing, feature extraction, model training, evaluation, model persistence, and prediction.

6. ORGANIZATION PROFILE

Codec Technologies is presented in the internship offer letter as a global platform focused on empowering learners and connecting diverse talent with career opportunities. The document states that Codec Technologies delivers dedicated IT and business consultancy services across **27+ countries**, with an emphasis on global innovation and strategic growth.

The internship program was structured around practical learning and assigned project or training tasks. According to the offer letter, the internship was conducted in a **Pan India, Hybrid** mode and interns were expected to report to assigned project head(s).

The organization provided an internship framework in which learning activities were combined with practical assignments. For an Artificial Intelligence intern, this type of structure provides an opportunity to move beyond theoretical definitions and understand how an AI/ML problem is broken into stages that can be implemented and evaluated.

The internship certificate and offer documentation identify the role as **Artificial Intelligence Intern**. The offer letter was issued on 17 July 2026 and identifies Dr. Anurag Shrivastava as Talent Acquisition Manager and the reporting structure as assigned project head(s).

The internship was therefore relevant to the student's Computer Science and Engineering curriculum because it connected programming, data processing, machine learning, and Natural Language Processing with a practical project.

7. INTERNSHIP OVERVIEW AND OBJECTIVES

7.1 Internship Overview

The internship was completed over a period of 45 days, from 17 July 2026 to 31 August 2026. Its primary focus was Artificial Intelligence, with the learning process covering the foundations required to understand, implement, and evaluate machine learning systems.

The training was intentionally broad rather than restricted to a single algorithm. It introduced the complete AI/ML lifecycle and the relationship between data, preprocessing, feature engineering, model selection, training, evaluation, and deployment-oriented thinking. This broad foundation was then connected to the practical Twitter Sentiment Analysis project.

7.2 Internship Objectives

- Develop a foundational understanding of Artificial Intelligence and Machine Learning.
- Understand supervised and unsupervised learning and their practical use cases.
- Strengthen Python programming skills for data-oriented and machine learning tasks.
- Learn how datasets are collected, loaded, inspected, cleaned, transformed, and prepared.
- Understand the role of feature engineering in machine learning.
- Develop introductory knowledge of Natural Language Processing for textual data.
- Understand common classification algorithms and when they are useful.
- Learn how to divide data into training and testing subsets and why evaluation on unseen data matters.
- Gain practical experience with Scikit-learn and common Python data science libraries.
- Understand how trained models can be saved and reused for future predictions.
- Apply the concepts through a complete Twitter sentiment classification project.

7.3 Expected Professional Development

In addition to technical knowledge, the internship was intended to strengthen the ability to approach a problem systematically. A machine learning task often appears simple from the outside, but implementation requires many small decisions: what data to use, how to represent it, how to handle noisy inputs, which model to choose, how to measure performance, and how to interpret the results.

The project helped reinforce this workflow because the final classifier depended not only on Logistic Regression but also on the quality of preprocessing and feature representation. This is an important practical lesson for future work in AI and ML.

8. TRAINING AND LEARNING DURING THE INTERNSHIP

8.1 Artificial Intelligence Fundamentals

The internship began with the broader concept of Artificial Intelligence. AI can be understood as the study and development of computational systems that perform tasks involving capabilities such as perception, pattern recognition, reasoning, prediction, language processing, and decision-making.

The training introduced the distinction between Artificial Intelligence, Machine Learning, and Deep Learning. Artificial Intelligence is the broadest concept. Machine Learning focuses on systems that learn patterns from data, while Deep Learning uses multi-layer neural network architectures to learn increasingly complex representations.

8.2 Machine Learning Concepts

Machine Learning was studied as a data-driven approach in which an algorithm learns from examples. The internship covered the idea that the quality of a model depends on both the algorithm and the data used to train it.

- **Supervised Learning:** learning from labeled examples where the desired output is known.
- **Unsupervised Learning:** finding structure or patterns in data without predefined output labels.
- **Reinforcement Learning:** learning through interaction with an environment using rewards and penalties.
- **Classification:** predicting a discrete category such as positive/negative or spam/not spam.
- **Regression:** predicting a continuous numerical value.

8.3 Machine Learning Workflow

A major learning outcome was understanding the typical machine learning workflow. Instead of treating model training as a single command, the internship emphasized the sequence of activities that makes a model meaningful and reproducible.

```
Problem Definition → Data Acquisition → Data Exploration → Data Cleaning → Feature Engineering → Train/Test Split → Model Selection → Training → Evaluation → Prediction → Persistence/Deployment
```

Each stage affects the next. Poor data preparation can reduce the usefulness of a strong algorithm, while a carefully designed feature representation can allow a relatively simple classifier to perform effectively.

8.4 Python Programming for AI/ML

Python was used as the primary programming language because of its extensive ecosystem for data analysis, scientific computing, machine learning, and Natural Language Processing. The training strengthened practical familiarity with variables, collections, functions, loops, modules, file handling, libraries, and notebook-based development.

8.5 Data Analysis and Preparation

Data analysis was treated as a necessary step before model training. Pandas provides DataFrame structures that make it possible to inspect datasets, select columns, check dimensions, examine missing values, and study target distributions.

The project directly applied these concepts to the Sentiment140 dataset. The dataset was loaded from a CSV file, column names were assigned, dimensions were inspected, initial records were displayed, missing values were checked, and the distribution of the target column was examined.

8.6 Natural Language Processing

Natural Language Processing became particularly important because the practical project used tweets as input. Human language is not naturally represented as the numerical matrix expected by most conventional machine learning algorithms. Text therefore needs to be cleaned and converted into numerical features.

The internship introduced common NLP preprocessing operations such as normalization, tokenization, stop-word removal, stemming, and feature representation. These operations reduce irrelevant variation in text and make the input more suitable for classification.

8.7 Classification Algorithms

The internship provided exposure to several commonly used classification approaches. Logistic Regression was especially relevant to the final project because it is computationally practical for large sparse text representations and provides a strong baseline for binary classification.

Other classification concepts included decision trees, k-nearest neighbors, support vector machines, Naive Bayes, and ensemble approaches. Understanding multiple algorithms is useful because different datasets have different characteristics and no single model is universally optimal.

8.8 Model Evaluation

Evaluation was studied as an essential part of machine learning. A model that produces predictions is not automatically a useful model. Its performance must be measured on data that was not used to fit the model.

- Accuracy: proportion of correct predictions.
- Precision: proportion of predicted positive cases that are actually positive.
- Recall: proportion of actual positive cases that are correctly identified.
- F1-score: harmonic combination of precision and recall.
- Confusion matrix: table showing the distribution of correct and incorrect class predictions.

The Twitter Sentiment Analysis project primarily used accuracy because the implemented notebook explicitly calculates training and test accuracy using Scikit-learn's **accuracy_score**.

8.9 Model Persistence and Reusability

Another practical concept was model persistence. Training a model can require significant computation, especially when processing large datasets. Saving the trained model makes it possible to load the model later and perform predictions without repeating the entire training process.

In the project, Python's Pickle module was used to save the trained Logistic Regression model as **trained_model.sav**. The saved file was subsequently loaded and used to predict sentiment for selected test samples.

8.10 Google Colab and Notebook-Based Development

The practical project was developed in Google Colab. A notebook environment is useful for machine learning experimentation because code, outputs, explanations, and intermediate results can be kept together. It also provides access to a hosted computational environment, which is particularly convenient when working with large datasets.

The project used Colab commands to install the Kaggle library, configure the Kaggle API credentials, download the dataset, extract the archive, and execute the complete preprocessing and training pipeline.

8.11 Problem-Solving Skills Developed

The internship also strengthened the habit of breaking technical problems into smaller stages. For example, sentiment analysis can initially sound like a single task: "classify a tweet." In implementation, that task becomes data acquisition, label preparation, text cleaning, feature extraction, classification, evaluation, and prediction.

This decomposition is useful beyond the specific project. It is a general engineering approach that can be applied to computer vision, recommendation systems, fraud detection, document classification, and many other AI applications.

9. TECHNOLOGIES AND TOOLS USED

Technology / Tool	Role in the Internship / Project
Python	Primary programming language for data processing and machine learning
Google Colab	Cloud notebook environment used to execute the project
Kaggle	Source platform used to obtain the Sentiment140 dataset
Pandas	CSV loading, DataFrame creation, inspection, and manipulation
NumPy	Numerical and array-oriented operations
NLTK	Stop-word corpus and Natural Language Processing utilities
Regular Expressions	Cleaning non-alphabetic characters from tweet text
PorterStemmer	Reducing words to stem forms
Scikit-learn	Machine learning, feature extraction, splitting, training, and evaluation
TfidfVectorizer	Converting processed text into TF-IDF numerical features
Logistic Regression	Binary sentiment classification model
accuracy_score	Measuring training and test classification accuracy
Pickle	Saving and loading the trained model

The combination of these tools created a complete, reproducible introductory NLP classification pipeline. Each library had a specific responsibility rather than being used simply because it is common in machine learning tutorials. Pandas handled the data structure, NLTK and regular expressions handled language preprocessing, Scikit-learn handled feature extraction and modeling, and Pickle handled model persistence.

10. PROJECT DESCRIPTION: TWITTER SENTIMENT ANALYSIS

10.1 Project Overview

Twitter Sentiment Analysis is a Natural Language Processing and Machine Learning task in which textual messages are classified according to the sentiment expressed in them. The project developed during the internship focuses on binary classification of tweets into negative and positive categories.

The system was implemented in Python using Google Colab. The Sentiment140 dataset was downloaded from Kaggle and processed using Pandas and NLTK. The processed tweets were transformed into numerical TF-IDF vectors, and a Logistic Regression model was trained to predict the sentiment label.

10.2 Problem Statement

Social media produces extremely large volumes of textual content. Reading and manually classifying every tweet is not practical at scale. An automated sentiment analysis system can assist in identifying broad positive or negative reactions in large collections of social media messages.

The problem addressed by the project is therefore:

Given the text of a tweet, automatically predict whether the tweet expresses a negative or positive sentiment.

10.3 Project Objectives

- Acquire the Sentiment140 dataset from Kaggle.
- Load and inspect the dataset using Pandas.
- Understand the distribution of sentiment labels.
- Convert the original positive label 4 into the binary label 1.
- Clean tweet text using regular expressions.
- Convert text to lowercase and split it into words.
- Remove English stop words using NLTK.
- Apply Porter stemming to the remaining words.
- Separate input text and target labels.
- Create an 80:20 stratified train-test split.
- Convert text into TF-IDF numerical features.
- Train a Logistic Regression classifier.
- Measure training and testing accuracy.
- Save the trained model and reload it for prediction.

10.4 System Architecture

The implemented system follows a sequential machine learning pipeline. Each stage transforms the data into a form that can be processed by the next stage. The architecture is intentionally straightforward, which makes it suitable as a baseline NLP classification system and easy to understand, reproduce, and extend.

Kaggle Dataset → CSV Extraction → Pandas DataFrame → Data Inspection → Label Transformation → Text Preprocessing → Stemmed Text → Train/Test Split → TF-IDF Vectorizer → Logistic Regression → Accuracy Evaluation → Saved Model → Prediction

10.5 Input and Output

Component	Description
Input	Tweet text
Preprocessing	Cleaning, lowercase conversion, stop-word removal, stemming
Feature Representation	TF-IDF numerical vector
Classifier	Logistic Regression
Output	Negative (0) or Positive (1)
Evaluation	Training and testing accuracy

10.6 Why Logistic Regression?

Logistic Regression is a practical baseline for binary text classification because it can work effectively with high-dimensional sparse feature matrices such as those produced by TF-IDF. It is comparatively simple to train and provides a useful reference point against which more complex models can later be compared.

For a project based on approximately 1.6 million tweets, computational practicality matters. A simpler model that can be trained efficiently provides a useful baseline before introducing deep learning or transformer-based methods.

11. DATASET AND DATA UNDERSTANDING

11.1 Sentiment140 Dataset

The project uses the **Sentiment140** dataset obtained through Kaggle. The dataset contains approximately **1.6 million tweets** and is designed for sentiment analysis and text classification.

The dataset provides a large collection of Twitter text paired with sentiment targets. This scale is useful for demonstrating the practical challenges involved in working with a large text corpus rather than a small classroom dataset.

Dataset Characteristic	Description
Dataset Name	Sentiment140
Approximate Records	1,600,000 tweets
Source	Kaggle
Domain	Twitter / social media text
Task	Sentiment classification
Original Negative Label	0
Original Positive Label	4
Project Labels	0 = negative, 1 = positive

11.2 Dataset Columns

The CSV file contains several fields associated with each tweet. In the project, meaningful column names were assigned to make the data easier to work with programmatically.

Column	Purpose
target	Sentiment label
id	Tweet identifier
date	Tweet date/time information
flag	Dataset flag field
user	Tweet user information
text	Actual tweet text used for sentiment classification

11.3 Loading the Dataset

The Kaggle library is installed in the Colab environment and the dataset is downloaded using the Kaggle API. The downloaded ZIP archive is extracted using Python's ZipFile module. The CSV file is then loaded with Pandas using ISO-8859-1 encoding.

```
Kaggle API → sentiment140.zip → extraction → training.1600000.processed.noemoticon.csv →  
Pandas DataFrame
```

11.4 Initial Data Inspection

The notebook checks the dataset shape and displays the first records. These checks establish whether the file was loaded correctly and provide an initial understanding of its structure.

12. DATA PREPROCESSING AND NLP PIPELINE

12.1 Importance of Preprocessing

Raw social media text is noisy. Tweets can contain punctuation, special characters, inconsistent capitalization, abbreviations, and many common words that provide limited value to a simple classifier. Preprocessing attempts to reduce this noise while retaining information that can help distinguish sentiment.

The project performs several preprocessing operations before feature extraction. The sequence is important because the output of one operation becomes the input to the next.

12.2 Checking Missing Values

The project uses Pandas' `isnull().sum()` operation to inspect missing data. This provides a quick check of whether any dataset columns contain null values that could interfere with later processing.

12.3 Checking Target Distribution

The distribution of the target column is examined using `value_counts()`. Understanding the number of examples in each class is important because classification performance can be misleading when one class is heavily overrepresented.

12.4 Converting Target Labels

Sentiment140 uses 0 for negative tweets and 4 for positive tweets. For the project, the positive class is converted from 4 to 1. This produces a clean binary target representation.

Original Target	Project Target	Meaning
0	0	Negative
4	1	Positive

12.5 Regular Expression Cleaning

The preprocessing function uses a regular expression to replace every character that is not an English alphabetic character with a space. This removes punctuation and many other non-alphabetic symbols from the input text.

The purpose is not to make the tweet linguistically perfect. Instead, it creates a standardized representation that can be processed by the following steps.

12.6 Lowercase Conversion

After character cleaning, the text is converted to lowercase. Without this step, the same word could appear as separate features because of capitalization differences, increasing unnecessary variation in the vocabulary.

12.7 Tokenization

The cleaned text is split into individual words. These tokens become the basic units used by the stop-word removal and stemming stages.

12.8 Stop-Word Removal

The project uses the English stop-word list provided by NLTK. Stop words are common words that often contribute relatively little to basic text classification. Removing them can reduce the number of features and focus the representation on more informative terms.

12.9 Porter Stemming

The project uses NLTK's **PorterStemmer**. Stemming attempts to reduce related word forms to a common stem. This can reduce vocabulary variation and simplify the representation supplied to the classifier.

The stemming pipeline is:

```
Raw Text → Regex Cleaning → Lowercase → Split into Words → Stop-Word Filtering → Porter Stemming → Joined Text
```

12.10 Applying the Preprocessing Function

The preprocessing function is applied to every tweet in the text column. The processed result is stored in a new column named **stemmed content**. This preserves the original tweet text while creating a separate representation for model training.

Processing the entire dataset is computationally significant because the operation is performed over approximately 1.6 million tweets. The notebook itself notes that this step can take substantial time in Google Colab.

12.11 Preprocessing Summary

Stage	Representation
Original	Original Twitter message
Cleaning	Alphabetic characters retained
Normalization	Lowercase text
Tokenization	Individual words
Stop-word removal	Common English stop words removed
Stemming	Words reduced to stem forms
Final	Space-separated stemmed text

The preprocessing pipeline is one of the most important components of the project. The final classifier receives numerical features derived from this processed representation rather than the original tweet text.

13. FEATURE EXTRACTION WITH TF-IDF

13.1 Why Text Must Become Numbers

Machine learning algorithms such as Logistic Regression operate on numerical feature matrices. A raw sentence cannot be directly passed to the classifier as a string. The text must therefore be represented numerically.

TF-IDF, or Term Frequency-Inverse Document Frequency, is a common technique for representing text. It assigns weights to terms based on their frequency in a document and how common or rare those terms are across the collection of documents.

13.2 Term Frequency

Term Frequency reflects how frequently a term occurs within a particular document. A word that appears multiple times in a tweet may receive greater importance within that tweet's representation.

13.3 Inverse Document Frequency

Inverse Document Frequency reduces the importance of terms that occur across a large number of documents. A term that appears in many documents provides less information for distinguishing one document from another than a relatively uncommon term.

13.4 TF-IDF in the Project

The project uses Scikit-learn's **TfidfVectorizer**. The vectorizer is fitted on the training text and then used to transform the test text. This produces numerical feature matrices suitable for Logistic Regression.

The implemented sequence is:

```
vectorizer = TfidfVectorizer()  
x_train = vectorizer.fit_transform(x_train)  
x_test = vectorizer.transform(x_test)
```

13.5 Avoiding Test-Data Leakage

Fitting the vectorizer only on the training data is an important part of the workflow. The test set should represent unseen information, so the vocabulary and weighting rules are learned from the training portion and then applied to the test portion.

13.6 Sparse Text Representation

A large text corpus can contain a very large vocabulary, while each individual tweet contains only a small number of those words. The resulting matrix is therefore sparse. This is a natural representation for conventional text classifiers.

13.7 Why TF-IDF is Useful

TF-IDF does not provide deep contextual understanding, but it captures useful statistical information about word importance. When combined with a linear classifier, it provides a strong and computationally practical baseline for many text classification problems.

14. MODEL DEVELOPMENT USING LOGISTIC REGRESSION

14.1 Preparing X and Y

Once preprocessing is complete, the stemmed tweet content becomes the input variable X, while the transformed target column becomes the output variable Y. This creates the basic supervised learning structure: the model receives tweet-derived features and learns to associate them with known sentiment labels.

14.2 Train-Test Split

The project uses Scikit-learn's `train_test_split()` with `test_size=0.2`. Therefore, approximately 80% of the examples are used for training and 20% for testing.

The split uses `stratify=y`, which helps preserve the class distribution across the two subsets. A fixed `random_state=2` is also used for reproducibility.

Parameter	Project Setting	Purpose
Training portion	80%	Model fitting
Testing portion	20%	Evaluation on unseen data
Stratification	Enabled	Preserve class distribution
Random state	2	Reproducible split

14.3 Logistic Regression Model

The classifier is created using Scikit-learn's LogisticRegression class. The project sets `max_iter=1000` to provide the optimization procedure with sufficient iterations for the high-dimensional text feature space.

The core implementation is:

```
model = LogisticRegression(max_iter=1000)
model.fit(x_train, y_train)
```

14.4 Learning a Decision Boundary

In binary classification, Logistic Regression estimates the probability associated with the classes based on the input features. During training, the model learns coefficients associated with the TF-IDF features and uses them to distinguish the two sentiment classes.

The model is not manually given rules such as “happy means positive” or “bad means negative.” Instead, it learns statistical relationships between features and labels from the training examples.

14.5 Training and Testing Predictions

After fitting, predictions are generated for the training matrix and compared with the known training labels. The same trained model is then used on the test matrix. The latter provides a more meaningful indication of generalization because those examples were not used to fit the classifier.

15. MODEL EVALUATION AND RESULTS

15.1 Evaluation Method

The project evaluates the trained Logistic Regression model using Scikit-learn's **accuracy_score**. Accuracy is the proportion of predictions that match the actual labels.

The notebook evaluates both training and testing data. Comparing these values helps determine whether the model behaves similarly on examples it has seen and examples that were held out during training.

Metric / Item	Result
Algorithm	Logistic Regression
Text representation	TF-IDF
Training/Test split	80% / 20%
Evaluation metric	Accuracy
Reported test accuracy	Approximately 77.66%

15.2 Interpretation of the Result

The reported test accuracy of approximately **77.66%** indicates that the classifier correctly predicts a substantial majority of examples in the held-out test set. The result demonstrates that conventional machine learning, when combined with appropriate text preprocessing and TF-IDF features, can provide a useful baseline for sentiment classification.

The result should nevertheless be interpreted in context. Accuracy alone does not explain which types of tweets are misclassified. A tweet containing sarcasm, irony, ambiguous wording, unusual spelling, or context-dependent meaning may remain difficult for a simple statistical text model.

15.3 Training Accuracy

The notebook also calculates training accuracy by predicting the labels of the training matrix. Training accuracy describes how well the fitted classifier performs on data that was used during model fitting.

15.4 Test Accuracy

Test accuracy is calculated using predictions made on the held-out test matrix. Because the test examples are not used during fitting, this value is more relevant when discussing the model's ability to generalize.

15.5 Evaluation Limitations

The current notebook primarily reports accuracy. A more complete evaluation would include precision, recall, F1-score, and a confusion matrix. These metrics could reveal whether one sentiment class is more difficult for the classifier than the other.

16. PREDICTION AND MODEL PERSISTENCE

16.1 Saving the Trained Model

After training, the Logistic Regression model is saved using Python's Pickle module. The notebook stores the trained classifier in a file named **trained_model.sav**.

The operation is:

```
filename = 'trained_model.sav'
pickle.dump(model, open(filename, 'wb'))
```

Saving the model is useful because training on a large dataset can require substantial computation. A persisted model can be loaded later for inference without repeating the training process.

16.2 Loading the Model

The notebook loads the saved classifier with Pickle and assigns it to **loaded_model**. This demonstrates that the trained model can be reused after the original training operation.

```
loaded_model = pickle.load(open('trained_model.sav', 'rb'))
```

16.3 Prediction on Test Samples

Selected examples from the test feature matrix are passed to the loaded model. The actual label is printed and the predicted label is generated. The prediction is then converted into a readable classification: negative tweet or positive tweet.

This is a simple inference workflow. The system has already completed the expensive training stage and can now use the learned model to classify available feature vectors.

16.4 Important Deployment Consideration

For a real application, the fitted TF-IDF vectorizer should also be saved. A new tweet must pass through exactly the same vocabulary and transformation process used during training. Saving only the classifier would not be enough to reproduce the original feature representation.

17. APPLICATIONS, LIMITATIONS AND FUTURE SCOPE

17.1 Potential Applications

- **Social media monitoring:** estimating broad positive and negative reactions in large collections of posts.
- **Customer feedback analysis:** automatically categorizing large volumes of textual comments.
- **Brand perception:** monitoring sentiment associated with a product, campaign, or service.
- **Market research:** supporting analysis of public reactions to topics or products.
- **Review classification:** adapting the same general pipeline to product or service reviews.
- **Support analytics:** identifying potentially negative feedback for further human review.

17.2 Limitations of the Current System

- The classifier uses only two classes: negative and positive. Neutral and mixed sentiment are not separately represented.
- TF-IDF is primarily statistical and does not provide deep contextual understanding.
- Sarcasm and irony can be difficult to identify from word frequency alone.
- The preprocessing removes non-alphabetic characters, which may also remove useful information such as emojis or expressive punctuation.
- Stemming can sometimes produce crude word forms that do not correspond to natural language words.
- The system does not explicitly model relationships between words beyond the features generated by the vectorizer.
- The project evaluates primarily with accuracy, so additional metrics would provide a more complete performance analysis.
- A saved Logistic Regression model alone is not sufficient for deployment; the same fitted preprocessing and vectorization pipeline must also be preserved.

17.3 Future Improvements

- Persist both the trained classifier and fitted TF-IDF vectorizer.
- Add precision, recall, F1-score, and confusion matrix analysis.
- Experiment with TF-IDF n-grams to capture short word combinations.
- Compare Logistic Regression with Naive Bayes and Support Vector Machine baselines.
- Experiment with word embeddings and neural networks.
- Explore LSTM or other recurrent architectures for sequential text modeling.
- Use transformer-based models such as BERT for richer contextual representation.
- Extend the classifier to neutral, mixed, or multi-emotion sentiment categories.
- Develop a simple web interface or API for real-time prediction.
- Perform error analysis to understand which types of tweets the current model classifies incorrectly.

17.4 Scalability Considerations

The dataset demonstrates an important engineering consideration: preprocessing itself can become expensive at scale. Applying stemming and stop-word filtering to approximately 1.6 million tweets requires considerable computation. A future implementation could optimize preprocessing, cache intermediate results, process the data in batches, or use more efficient vectorized pipelines.

18. LEARNING OUTCOMES

The internship provided both conceptual and practical learning. The following outcomes summarize the main skills developed through the training and project implementation.

Learning Area	Outcome
Artificial Intelligence	Understanding AI as a broad field and identifying major application areas
Machine Learning	Understanding learning from data and the major categories of ML
Supervised Learning	Understanding labeled-data classification workflows
Python	Using Python libraries and notebook-based development for ML
Data Handling	Loading, inspecting, transforming, and preparing datasets
Pandas	Working with CSV data and DataFrames
NumPy	Working with numerical data and arrays
NLP	Understanding preprocessing of human language
Stop Words	Removing common low-information terms
Stemming	Reducing related word forms to stems
Feature Engineering	Converting text into useful numerical features
TF-IDF	Using term weighting for text classification
Classification	Building a binary text classifier
Logistic Regression	Training and applying a practical classification model
Evaluation	Measuring model performance using accuracy
Model Persistence	Saving and reusing trained ML models
Problem Solving	Breaking an AI task into a complete implementation pipeline

18.1 Technical Learning

The most significant technical learning was understanding how individual components of a machine learning project fit together. Before practical implementation, concepts such as preprocessing, vectorization, classification, and evaluation can appear as separate topics. The project connected them into a single workflow in which every stage has a specific purpose.

Working with text also demonstrated that the quality of the input representation is critical. Logistic Regression itself is only one part of the system. Without preprocessing and TF-IDF vectorization, the algorithm would not receive useful numerical input.

18.2 Conceptual Learning

The internship strengthened understanding of why training and testing data should be separated, why test data should remain unseen during model fitting, and why evaluation metrics are necessary. It also introduced the idea that model performance should be interpreted in the context of its dataset, preprocessing choices, and application requirements.

18.3 Engineering Learning

The project provided exposure to reproducibility and reuse. A fixed random state makes the train-test split reproducible, while saving the trained model makes the classifier reusable. These details are small but important when moving from an experimental notebook toward a more reliable machine learning application.

19. INTERNSHIP EXPERIENCE AND PROFESSIONAL DEVELOPMENT

19.1 Bridging Theory and Practice

One of the main benefits of the internship was the opportunity to connect theoretical machine learning concepts with implementation. Definitions of supervised learning, classification, feature engineering, and evaluation become more meaningful when they are used to solve an actual problem.

The Twitter Sentiment Analysis project provided a clear example of this transition. A theoretical statement such as machine learning learning patterns from data becomes concrete when a classifier is trained on labeled tweets and then asked to predict labels for unseen tweets.

19.2 Working with Large-Scale Text

The Sentiment140 dataset provided exposure to the practical challenges of processing a large text corpus. The preprocessing function is applied to every tweet, and the notebook notes that this operation can take substantial time in the Colab environment. This demonstrated that computational cost is not limited to model training; preprocessing can also become a major part of an ML workflow.

19.3 Understanding Model Limitations

The internship also helped establish realistic expectations about machine learning models. An accuracy value does not mean that a model understands language in a human-like way. The current classifier identifies statistical patterns in processed text. It does not reason about sarcasm, world knowledge, or conversational context.

19.4 Career Relevance

The skills developed during the internship form a useful foundation for future work in Machine Learning and Artificial Intelligence. The project can be extended into more advanced NLP systems, deployment projects, data science applications, or deep learning experiments.

19.5 Professional Skills

- Breaking a large technical task into smaller implementation stages.
- Reading and understanding machine learning notebooks and library documentation.
- Working with data before attempting model training.
- Evaluating results instead of assuming that a trained model is automatically successful.
- Keeping preprocessing and model steps consistent between training and prediction.
- Thinking about computational cost when working with large datasets.
- Documenting technical decisions and project workflow in a structured report.

These professional habits are transferable to other software and AI projects. In particular, the ability to understand the entire data-to-prediction pipeline is more valuable than memorizing isolated library commands.

20. CONCLUSION

The 45 Days Artificial Intelligence Internship at **Codec Technologies Pvt. Ltd.** provided a broad introduction to Artificial Intelligence and Machine Learning. The internship was conducted from **17 July 2026 to 31 August 2026** in a **Pan India, Hybrid** mode, with the designation of **Artificial Intelligence Intern**.

The training developed an understanding of the fundamental AI/ML workflow, including data handling, preprocessing, supervised learning, classification, Natural Language Processing, feature engineering, model training, evaluation, and model persistence. These topics provided the foundation required to understand how an AI problem can be transformed into an implementable machine learning system.

The practical project, Twitter Sentiment Analysis, applied these concepts to the Sentiment140 dataset containing approximately 1.6 million tweets. The project converted the original sentiment labels into a binary representation, cleaned and normalized the tweet text, removed English stop words, applied Porter stemming, and transformed the processed text into TF-IDF features.

A Logistic Regression classifier was then trained using an 80:20 stratified train-test split. The model was evaluated using accuracy, with the reported test accuracy being approximately **77.66%**. The trained model was subsequently saved using Pickle and loaded again to perform predictions on selected examples.

The project demonstrates that a relatively simple machine learning pipeline can provide useful results when supported by appropriate data preprocessing and feature engineering. At the same time, its limitations highlight the need for more advanced approaches when greater contextual understanding is required.

Overall, the internship served as a bridge between academic concepts and practical implementation. It established a foundation for further study in Natural Language Processing, Deep Learning, transformer-based models, model deployment, and broader AI engineering.

21. INTERNSHIP DOCUMENTATION AND REFERENCES

21.1 Internship Offer Letter

The offer letter issued to Shubham Mishra confirms a 45 Days internship at Codec Technologies and identifies the designation as **Artificial Intelligence Intern**. It specifies **Pan India, Hybrid** as the location and mode, the internship period as **17/07/2026 to 31/08/2026**, and reporting to assigned project head(s).

The offer letter describes Codec Technologies as a global platform dedicated to empowering learners and connecting diverse talent with career opportunities. It states that the organization provides IT and business consultancy services across **27+ countries**. The document also states that the program provides an immersive experience intended to develop industry-level knowledge and valuable skills.

Particular	Details
Student Name	Shubham Mishra
University	IILM University, Greater Noida, U.P.
School	School of Computer Science and Engineering
Degree	B.Tech (Computer Science and Engineering)
Session	2025–2029
Organization	Codec Technologies Pvt. Ltd.
Designation	Artificial Intelligence Intern
Duration	45 Days
Internship Dates	17 July 2026 – 31 August 2026
Mode	Pan India, Hybrid
Reporting	Assigned Project Head(s)
NCS ID	E19E86-0116588288923
AICTE ID	CORPORATE6759d549ce59e1733940553
Date of Issue	17 July 2026

21.2 Technical References

- Sentiment140 dataset, obtained through Kaggle and used as the dataset for the Twitter sentiment classification project.
- Python documentation, for the programming language and standard library concepts.
- Pandas documentation, for DataFrame operations and CSV data handling.
- NumPy documentation, for numerical computing and array operations.
- NLTK documentation, for stop-word processing and Porter stemming.
- Scikit-learn documentation, for train_test_split, TfidfVectorizer, Logistic Regression, and accuracy_score.
- Google Colab, used as the cloud notebook development and execution environment.

21.3 Project Notebook

The practical implementation was developed in a Google Colab notebook titled **Twitter Sentiment Analysis AI ML**. The notebook contains the sequence from dataset download and extraction through preprocessing, model training, evaluation, model saving, and prediction.

22. APPENDIX A: PROJECT IMPLEMENTATION OUTLINE

This appendix summarizes the implemented notebook in logical stages. It is included to make the technical structure of the project clear without reproducing the entire source code.

Stage	Implementation
Environment setup	Install Kaggle package and configure Kaggle API
Dataset acquisition	Download Sentiment140 ZIP from Kaggle
Extraction	Extract the CSV using ZipFile
Libraries	Pandas, NumPy, re, NLTK, Scikit-learn, Pickle
Dataset loading	Read the Sentiment140 CSV with ISO-8859-1 encoding
Column naming	Assign target, id, date, flag, user, text
Inspection	Shape, head, missing values, target counts
Label conversion	Replace target 4 with 1
Text preprocessing	Regex, lowercase, tokenization, stop-word removal, stemming
Input/label separation	X = processed text, Y = target
Split	80/20 stratified train-test split
Vectorization	TfidfVectorizer
Model	LogisticRegression(max_iter=1000)
Training	model.fit(x_train, y_train)
Evaluation	accuracy_score on training and test predictions
Persistence	Pickle model to trained_model.sav
Inference	Load saved model and predict selected test examples

22.1 Reproducibility

The project uses a fixed random state during train-test splitting. Reproducibility is important in machine learning because different random splits can produce different results. Fixing the split makes it easier to compare changes to preprocessing or model configuration.

22.2 Core Design Principle

The project demonstrates a useful baseline design principle: keep each stage of the machine learning pipeline explicit. Data preparation, feature extraction, model training, evaluation, and prediction are separate conceptual steps. This separation makes the workflow easier to debug and extend.

23. APPENDIX B: DETAILED PROJECT WORKFLOW

23.1 Phase 1: Data Acquisition

The Kaggle API is used to download the Sentiment140 dataset into the Colab environment. The ZIP archive is extracted so that the CSV training file can be read by Pandas.

23.2 Phase 2: Data Understanding

The dataset is inspected before transformation. Shape, sample records, missing values, and target distribution are checked. This phase verifies the dataset structure and establishes the initial understanding required for preprocessing.

23.3 Phase 3: Label Preparation

The original positive label 4 is converted into 1. The resulting binary representation is easier to use in the Logistic Regression classification workflow.

23.4 Phase 4: NLP Preprocessing

Each tweet is cleaned and normalized. Non-alphabetic characters are replaced with spaces, text is converted to lowercase, tokens are created, English stop words are removed, and the remaining words are stemmed with PorterStemmer.

23.5 Phase 5: Dataset Split

The processed text and labels are separated into training and testing portions. Stratification preserves the target distribution, while the fixed random state supports reproducibility.

23.6 Phase 6: Numerical Representation

TF-IDF vectorization converts the processed tweets into sparse numerical matrices. The vectorizer learns its representation from the training data and then transforms the test data using the same learned representation.

23.7 Phase 7: Model Training

Logistic Regression is fitted on the TF-IDF training matrix. The classifier learns coefficients that help separate positive and negative tweets.

23.8 Phase 8: Evaluation

Predictions are generated for both training and testing data. Accuracy is calculated for each set, with the project reporting approximately 77.66% test accuracy.

23.9 Phase 9: Model Reuse

The classifier is saved to disk and loaded again. Selected test vectors are passed to the loaded classifier to demonstrate sentiment prediction.

24. APPENDIX C: FUTURE EXTENSION DESIGN

A future application based on this project could be structured as a small NLP prediction service. The architecture would contain a preprocessing component, the persisted TF-IDF vectorizer, the persisted Logistic Regression model, and an application interface.

A possible extension would follow:

```
User Tweet → Input Validation → Same Text Preprocessing → Saved TF-IDF Vectorizer → Saved  
Logistic Regression Model → Sentiment Probability/Label → User Interface
```

The preprocessing used during inference must match the preprocessing used during training. Otherwise, new text could be represented differently from the data on which the classifier learned its coefficients.

The project could also be expanded into a comparative study. Logistic Regression could serve as the baseline while alternative models are trained using the same dataset and evaluation protocol. This would make it possible to compare accuracy, training time, inference time, memory requirements, and robustness.

A more advanced version could use contextual language models. Such models can represent relationships between words and may handle context more effectively than TF-IDF, although they introduce greater computational and deployment complexity.

This progression illustrates a useful development path: establish a reliable baseline first, measure it carefully, understand its failure cases, and then introduce additional complexity only where it addresses a demonstrated limitation.

END OF INTERNSHIP REPORT

Shubham Mishra
B.Tech (Computer Science and Engineering)
IILM University, Greater Noida, U.P.
Session 2025–2029